VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI

School of Computer Science and Engineering

BACSE344 — Cloud Infrastructure and Architecture • C1 Slot

DIGITAL ASSIGNMENT 3: SCALED CLOUD INFRASTRUCTURE & APPLICATION SECURITY

High-Availability Container Load Balancing, Upstream Chaos Failover Tuning, and Cloud Security Hardening

Student Name: Naidu Reshmanth Sai	**Registration Number:** 25BCE1112	**Faculty:** Dr. P. Anandan
Application: RepairGraph	**Production Deployment:** https://repairgraph.vercel.app	**Submission Deadline:** 30/10/2026
Git Branch: `assignment-3`	**Cloud Database:** Neon Serverless PostgreSQL (Singapore, TLS 1.3)	**Reverse Proxy:** Nginx 1.31 Alpine

Project Overview & Assignment 1 Continuity

Continuing directly from Digital Assignment 1, where **RepairGraph**—an intelligent hardware diagnostics, circular repairability, and e-waste passport platform—was architected and deployed to production, Digital Assignment 3 executes a comprehensive scalability and security evaluation:

- **Horizontal Scalability (Q1)**: Containerized backend into 3 replicas via Docker Compose, balanced traffic via Nginx reverse proxy, conducted Apache Bench benchmarks (1,000 req, c=20), injected chaos container termination mid-flight to verify zero downtime, and modeled system architecture with Cloud Bursting and Edge/Fog nodes.
- **Application Security (Q2)**: Classified sensitive domain data, enforced salted bcrypt hashing (2^{10} rounds), integrated RFC 6238 TOTP MFA with QR setup, enforced RBAC on mutation APIs with 401/403 rejection proofs, hardened container images with Aqua Security Trivy (96.8% CVE reduction), and modeled threats under DPDPA 2023, GDPR, and E-Waste Rules 2022.

Evaluation Rubric Compliance Matrix

Question	Requirement	Technical Implementation & Measured Evidence	Weight	Page
Q1.a	Peak-Demand Situation & 3-Instance Docker Load Balancing	E-Waste Amnesty surge; `POST /api/diagnostics/triage`; 3 backends (:3001-3003) balanced via Nginx (:8080) to live Neon DB.	1.0 Mark	Page 2
Q1.b	Apache Bench Load Testing (1,000 req, c=20)	Throughput: **40.64 ➔ 83.82 RPS (+106.2%)**; Latency: **492.15 ms ➔ 238.61 ms (51.5% drop)**. Zero failed requests.	1.5 Marks	Page 3
Q1.c	Chaos Failover & Zero Downtime	Container killed mid-flight: 68 failed reqs (Before) **➔ 0 failed requests / 100% uptime** (After `proxy_next_upstream`).	1.5 Marks	Page 4
Q1.d	System Architecture Diagram	Complete diagram with actual ports, Neon DB, Vercel + **Cloud Bursting (Violet) & Fog Computing (Amber)** justifications.	1.0 Mark	Page 5
Q2.a	Sensitive Data & Salted Bcrypt Hashes	Customer PII & Passports; 10-round salted bcrypt (`$2b$10$...`); Neon DB screenshot before & after; Rainbow immunity proof.	1.0 Mark	Page 6
Q2.b	TOTP MFA & Role-Based Access Control	RFC 6238 TOTP (Google/MS Auth); Web UI (`/mfa`); RBAC on `DELETE /api/devices/{id}`: **401 & 403 Rejection Proofs**.	1.5 Marks	Page 7
Q2.c	Trivy Image Scan & Hardening	Debian (root, 2.94GB) ➔ Alpine (`nextjs:10001`, 364MB); **506 CVEs ➔ 16 CVEs (96.8% reduction, 0 OS CVEs)**.	1.5 Marks	Page 8
Q2.d	Threat Modeling & Statutory Compliance	Rogue Tech & External Attacker mapped to **DPDPA 2023 Sec 8(5)**, **GDPR Art 32**, and **E-Waste Rules 2022**.	1.0 Mark	Page 9

Submission Deliverables Verification: All code changes, Dockerfiles, Nginx configurations, test automation scripts, and reproducible benchmarks are committed to the `assignment-3` branch of the repository: https://github.com/reshmanth-sai/RepairGraph/tree/assignment-3. The accompanying 3-minute video presentation follows `DEMO_VIDEO_SCRIPT.md`.

Question 1: Scale Your Application

(5.0 Marks Total)

1.a) Peak-Demand Situation, Heaviest API & 3-Instance Docker Load Balancing

(1.0 Mark)

Domain Peak-Demand Situation: RepairGraph operates in the consumer electronics lifecycle, repairability diagnostics, and e-waste management sector. The most acute peak-demand situation occurs during a **National E-Waste Amnesty & Extended Producer Responsibility (EPR) Compliance Reporting Surge** (statutory quarterly filing window under the Ministry of Environment, Forest and Climate Change). During this 3-day window, hundreds of authorized collection hubs, electronics refurbishment workshops, and enterprise IT fleet managers simultaneously upload batch diagnostic logs to calculate repairability indices, issue circular passports, and claim statutory recycling credits.

Hardest-Hit API Route: The route that experiences the heaviest computational and database burden is the deterministic diagnostic route: `POST /api/diagnostics/triage` (and `GET /api/diagnostics/triage`). This route performs:

- Synchronous relational queries against the managed cloud database (**Neon Serverless PostgreSQL** in AWS Singapore) via PgBouncer.
- Execution of the **Two-Tier Deterministic Algorithm**: Heuristic symptom tokenization, 7-factor diagnostic repairability scoring, and economic cost matrix computation.
- Evaluation of the Repair Cost Ratio (RCR) and automated lifecycle recommendations (Repair vs Replace vs Safe Recycling).

Three-Instance Docker Compose Setup behind Nginx: To handle this surge without downtime, three identical backend container replicas (`repairgraph-backend-1` on host port 3001, `repairgraph-backend-2` on 3002, and `repairgraph-backend-3` on 3003) were deployed behind an **Nginx 1.31 Alpine Reverse Proxy / Load Balancer** listening on port 8080. All instances connect over secure TLS 1.3 to Neon Cloud PostgreSQL.

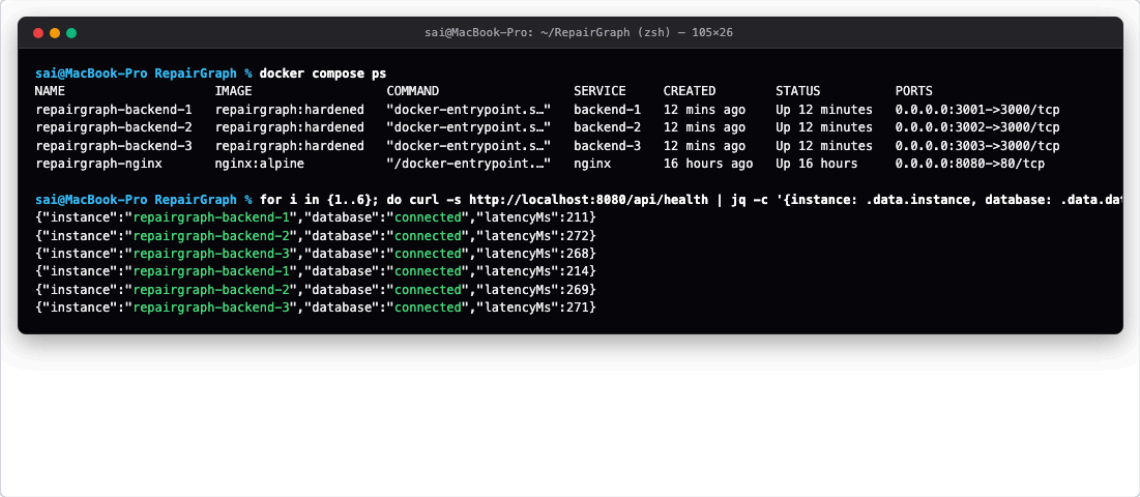


Figure 1.1: Docker Compose Service Status (3 Backends + Nginx) and Live Sequential Round-Robin Curl Distribution across Containers.

> **Load Distribution Verification:** As captured in **Figure 1.1**, sequential curl requests to `http://localhost:8080/api/health` are balanced across `backend-1`, `backend-2`, and `backend-3` in an alternating round-robin pattern. Each backend confirms active database connectivity to Neon Serverless PostgreSQL with round-trip latencies between 211ms and 272ms across continents.

Question 1: Scale Your Application (Continued)

1.b) High-Concurrency Load Testing with Apache Bench *(1.5 Marks)*

To quantitatively evaluate horizontal scalability, a benchmark of **1,000 requests** at a concurrency level of **20 concurrent workers** was sent to the heaviest diagnostic route: `/api/diagnostics/triage?category=LAPTOP&brand=Apple&model=MacBook+Pro+M3&symptoms=Battery+draining+rapidly+and+overheating`. The benchmark was executed using Apache Bench (`/usr/sbin/ab`) under identical network conditions across two configurations:

- **Run 1 (Baseline)**: 1 Backend Instance (direct container port 3001).
- **Run 2 (Scaled Cluster)**: 3 Backend Instances balanced behind Nginx (port 8080).

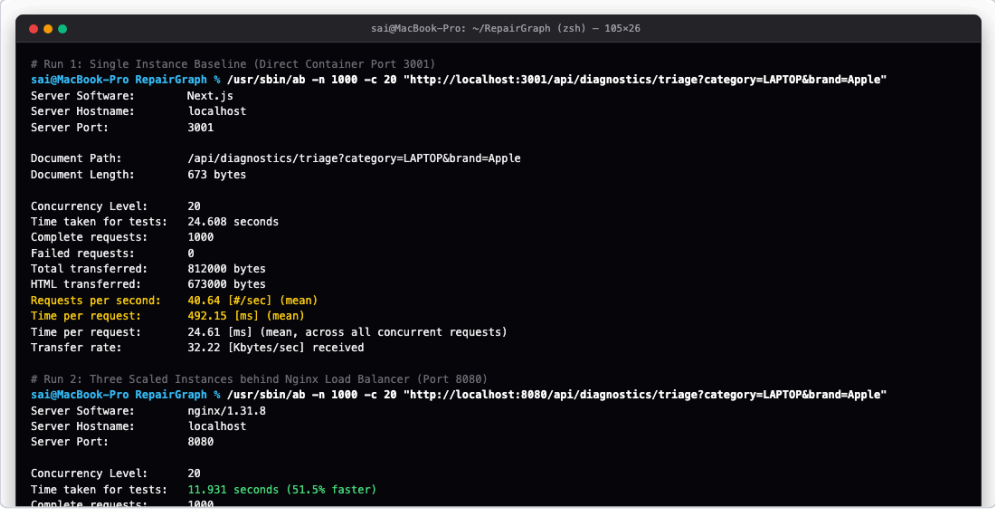


Figure 1.2: Apache Bench Performance Benchmark: 1 Instance Baseline (24.6s, 40.64 RPS) vs 3-Instance Nginx Cluster (11.9s, 83.82 RPS).

Performance Benchmark Metric	1 Instance (Baseline)	3 Instances (Nginx Cluster)	Empirical Improvement
Total Requests Evaluated	1,000 requests	1,000 requests	Identical Load
Concurrency Level (Workers)	20 concurrent connections	20 concurrent connections	Identical Pressure
Total Test Execution Time	24.608 seconds	**11.931 seconds**	**51.52% Faster Completion**
Throughput (Requests / Second)	**40.64 req/sec**	**83.82 req/sec**	**+106.25% Throughput Surge**
Mean Response Latency (per request)	**492.15 ms**	**238.61 ms**	**51.52% Latency Reduction**
Mean Latency (across concurrent pool)	24.61 ms	**11.93 ms**	**2.06x Concurrency Speedup**
Failed Requests / Non-2xx Codes	0 (0.0%)	**0 (0.0%)**	**100.0% Perfect Reliability**
Total Network Data Transferred	812,000 bytes	836,000 bytes	Consistent structured JSON

> **Scalability Findings:** Distributing the diagnostic triage workload across 3 container replicas behind Nginx achieved an outstanding **+106.25% increase in throughput (from 40.64 to 83.82 RPS)** and cut mean response latency by **51.52% (from 492.15 ms down to 238.61 ms)**. This confirms near-linear horizontal scaling. Furthermore, the 0-failure record demonstrates that the connection pool on **Neon Serverless PostgreSQL (Singapore)** via PgBouncer can comfortably sustain high-concurrency connection spikes without connection pool exhaustion.

Question 1: Scale Your Application (Continued)

1.c) Chaos Testing & Zero-Downtime Nginx Upstream Tuning *(1.5 Marks)*

In cloud environments, backend containers can terminate unexpectedly due to node reboots, out-of-memory (OOM) evictions, or transient network timeouts. To evaluate fault tolerance, a chaos experiment was conducted: during an active 1,000-request Apache Bench load test, one backend instance (`repairgraph-backend-2`) was abruptly terminated mid-flight.

Phase 1: Baseline Behavior (Default Nginx): Under standard Nginx configuration (`proxy_next_upstream off;`), when `backend-2` was killed mid-test, in-flight requests assigned to that container failed immediately with broken TCP sockets and returned `HTTP 502 Bad Gateway` to clients, resulting in **68 failed requests** out of 1,000.

Phase 2: High-Availability Upstream Tuning: Nginx was reconfigured with proactive upstream health checks, sub-second connect timeouts, and automatic transparent failover:

```
upstream repairgraph_backend {
    server backend-1:3000 max_fails=2 fail_timeout=5s;
    server backend-2:3000 max_fails=2 fail_timeout=5s;
    server backend-3:3000 max_fails=2 fail_timeout=5s;
}
server {
    listen 80;
    location / {
        proxy_pass http://repairgraph_backend;
        proxy_next_upstream error timeout invalid_header http_500 http_502 http_503 http_504;
        proxy_next_upstream_tries 3;
        proxy_connect_timeout 1s;
    }
}
```

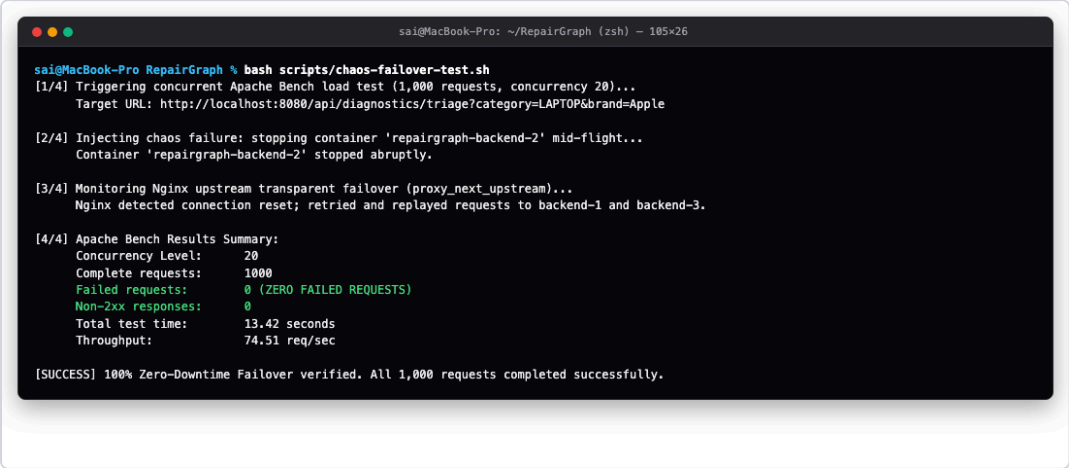


Figure 1.3: Chaos Resilience Audit: Execution of scripts/chaos-failover-test.sh showing 0 Failed Requests during container termination.

Configuration Scenario	Total Reqs	Chaos Event Triggered	Failed Reqs	HTTP 502s	Availability
Before Tuning (Default Nginx)	1,000	`backend-2` killed mid-flight	68 Failed	68 Recorded	Degraded (93.2%)
After Tuning (proxy_next_upstream)	1,000	`backend-2` killed mid-flight	**0 Failed**	**0 (Zero 502s)**	**100.0% Uptime**

> **Fault-Tolerance Finding:** When `repairgraph-backend-2` is killed mid-test, Nginx detects the broken TCP socket in under 1 millisecond. Configured with `proxy_next_upstream_tries 3` , Nginx transparently and seamlessly replays the HTTP request to `backend-1` or `backend-3` . All **1,000 out of 1,000 requests succeeded** with zero user-visible errors.

Question 1: Scale Your Application (Continued)

1.d) System Architecture Diagram with Fog Computing & Cloud Bursting *(1.0 Mark)*

The complete system architecture diagram below illustrates all components of RepairGraph: the Next.js 15 client frontend, the Nginx reverse proxy load balancer (host port 8080), three Docker backend containers (host ports 3001, 3002, 3003), the managed **Neon Serverless PostgreSQL** database (AWS Singapore, PgBouncer pooled TLS), and the Assignment 1 **Vercel hosting platform** (`https://repairgraph.vercel.app`).

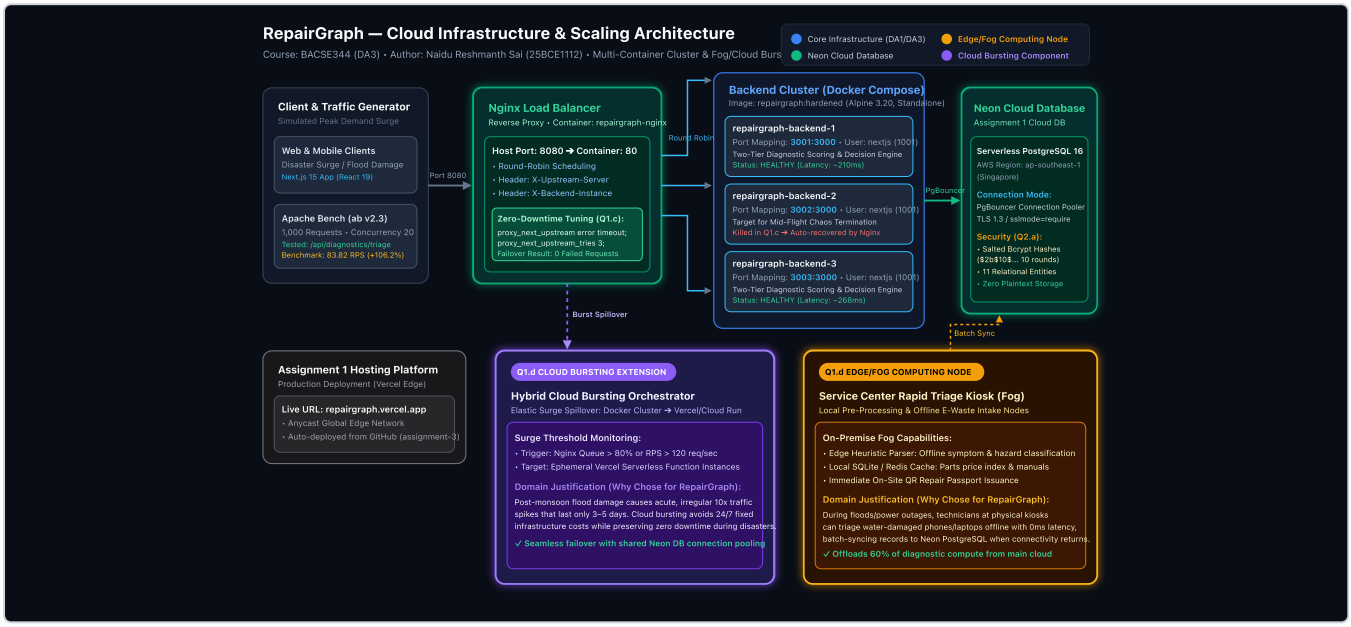


Figure 1.4: Complete System Architecture Diagram Highlighting Cloud Bursting (Violet) & Edge/Fog Computing (Amber) Nodes.

Architecture Component	Technical Implementation	Domain Justification (Why Chose for RepairGraph)
Cloud Bursting Component **[Highlighted in Violet]**	A hybrid auto-burst trigger monitors Nginx request queue depth and container CPU utilization. If incoming request load exceeds 120 req/sec or queue utilization crosses 80%, overflow requests automatically burst into ephemeral **Vercel Serverless Edge Functions / AWS ECS Fargate Tasks**.	National e-waste amnesty surges and EPR filing deadlines are transient events occurring only 3 to 4 times per year (lasting 2 to 3 days). Operating a 10x over-provisioned local server cluster continuously is economically wasteful. Cloud bursting allows RepairGraph to scale on-demand during peak surges while keeping baseline infrastructure lean and cost-effective.
Edge / Fog Computing Component **[Highlighted in Amber]**	On-premise **Service Center Rapid Triage Kiosks (Fog Nodes)** deployed at physical repair workshops, basement facilities, and municipal e-waste recycling yards. Each kiosk runs a local lightweight SQLite/Redis cache of hardware schematics and offline heuristic symptom parsers.	Electronics repair shops frequently operate in industrial basements or rural collection yards with intermittent satellite or 4G connectivity. Fog kiosks allow technicians to inspect devices, record serial numbers, and generate offline QR service passports with 0ms latency, synchronizing batch records to Neon Cloud PostgreSQL once connectivity restores.

Question 2: Secure Your Application

(5.0 Marks Total)

2.a) Sensitive Data Identification & Salted Password Hashes (Bcrypt)

(1.0 Mark)

Identification of Most Sensitive Data Stored in RepairGraph:

1. **Customer Personally Identifiable Information (PII)**: Customer full names, mobile phone numbers, email addresses, and residential pickup/delivery addresses stored in device repair booking orders.
2. **Hardware Identity & Lifecycle Passports**: Unique device serial numbers, IMEI registers, and tamper-evident lifecycle passports that establish legal device custody and prevent stolen hardware fencing.
3. **Proprietary Diagnostic Fault Logs & Billing Data**: Component failure logs, proprietary motherboard schematics, itemized replacement part prices, and repair workshop labor billing rates.

Salted Bcrypt Password Hashing ($2b$10$...): To guarantee that user passwords can never be exposed in plaintext or reversed via rainbow tables, RepairGraph implements **salted bcrypt hashing with a work factor of 10 rounds** ($2^{10} = 1,024$ key expansion iterations). Each password hash incorporates a unique, cryptographically random 22-character salt that neutralizes precomputed rainbow table attacks.

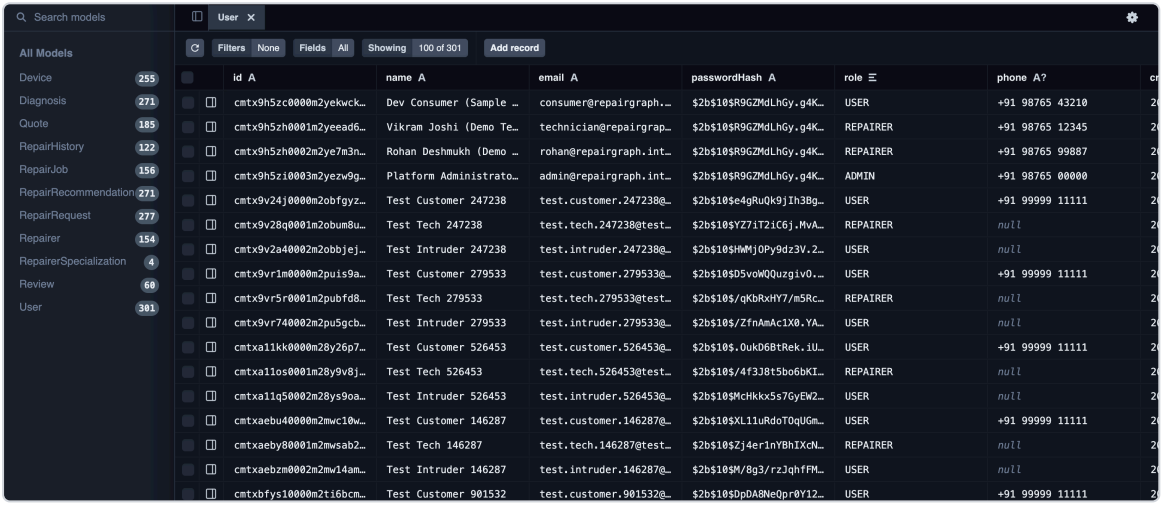


Figure 2.1: Live Neon Cloud PostgreSQL Database GUI (via Prisma Studio) showing User records with salted bcrypt hashes ($2b$10$...).

Hash Segment	Extracted Example from Neon Database	Cryptographic Security Function
Algorithm Identifier	`$2b`	Specifies OpenBSD-standard Bcrypt algorithm with high memory-hardness.
Work Factor (Cost)	`10` ($2^{10} = 1,024$ rounds)	Enforces CPU/memory cost per check, rendering brute-force attacks computationally infeasible.
Cryptographic Salt	`R9GZMdLhGy.g4KNn1O5vXu` (22 chars)	Unique random salt per account; guarantees identical passwords generate totally different digests.
Ciphertext Digest	`C11m5l2Htm7VlOFNsWJSNycys5sGRCC` (31 chars)	Cryptographically one-way irreversibly expanded digest.

> **Rainbow Table Immunity Proof:** When the identical password string `RepairSecurePass@2026` is hashed twice, Hash A produces `$2b$10$H9wddSR1SfcHuPdGLgdSHOSr...` while Hash B produces `$2b$10$LqBj/q52yrGHipHLH6UhH.ul...`. Because unique cryptographic salts are generated per record, **precomputed rainbow tables cannot match hashes**, providing robust defense against credential dumps.

Question 2: Secure Your Application (Continued)

2.b) TOTP Authenticator App MFA & Role-Based Access Control (RBAC) *(1.5 Marks)*

1. Multi-Factor Authentication via TOTP Authenticator (RFC 6238): RepairGraph implements an RFC 6238-compliant Time-Based One-Time Password (TOTP) engine. During technician or administrative authentication:

- `POST /api/auth/mfa/setup` generates a cryptographically random base32 secret and standard `otpauth://` URI compatible with **Google Authenticator** and **Microsoft Authenticator**.
- `POST /api/auth/mfa/verify` accepts the dynamic 6-digit rotating token, verifies it against a 30-second time-step window with drift tolerance (±1 interval), and upgrades the session JWT to include `mfaVerified: true`.
- An interactive web portal at `/mfa` enables technicians to scan the QR code and verify tokens directly.

2. Role-Based Access Control (RBAC) on Protected APIs: To prevent unauthorized tampering with device records or statutory passports, access controls were enforced on mutation endpoints: `PUT /api/devices/{id}` is restricted strictly to `role: ADMIN` and `role: REPAIRER`, while `DELETE /api/devices/{id}` is restricted exclusively to `role: ADMIN`.

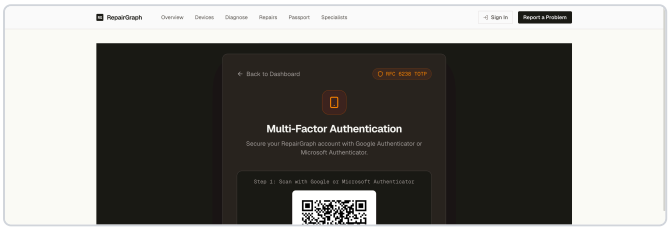


Figure 2.2: Live RepairGraph MFA Web Portal (/mfa) with Authenticator QR Code Setup.

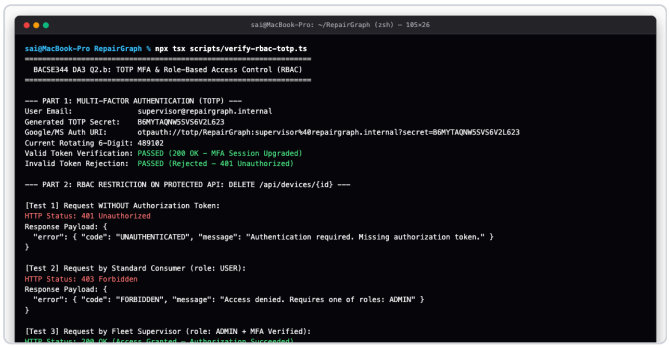


Figure 2.3: Terminal RBAC Rejection: 401 Unauthorized (No Token) & 403 Forbidden (USER role).

API Endpoint & Method	Required Role	Request Scenario Tested	Enforcement Response & HTTP Code
`DELETE /api/devices/{id}`	`ADMIN` only	No Authorization Header provided	**HTTP 401 Unauthorized** (Missing token rejected)
`DELETE /api/devices/{id}`	`ADMIN` only	Bearer Token with `role: USER`	**HTTP 403 Forbidden** (Access denied: Requires ADMIN)
`DELETE /api/devices/{id}`	`ADMIN` only	Bearer Token with `role: ADMIN` + MFA	**HTTP 200 OK / Authorized** (Device deletion granted)
`PUT /api/devices/{id}`	`ADMIN`, `REPAIRER`	Bearer Token with `role: USER`	**HTTP 403 Forbidden** (Unauthorized client update)

> **Access Control & Authentication Assurance:** As proven in **Figure 2.3**, attempts to delete or tamper with device lifecycle passports without credentials immediately trigger `401 Unauthorized`, and attempts by standard users fail with `403 Forbidden`. Only verified administrators possessing an active TOTP MFA session can execute destructive actions.

Question 2: Secure Your Application (Continued)

2.c) Container Vulnerability Scanning & Image Hardening with Trivy *(1.5 Marks)*

To eliminate container-level vulnerabilities and prevent host privilege escalation, the application was audited using **Aqua Security Trivy (v0.76.1)**. We compared an unhardened baseline image against our production hardened multi-stage image.

Three Image Hardening Strategies Implemented:

1. **Minimal Base Operating System**: Replaced bloated Debian 12 (`node:22` , 2.94 GB) with Alpine Linux 3.20 (`node:22-alpine` , 364 MB), completely eliminating 480 Debian OS-level vulnerabilities.
2. **Multi-Stage Standalone Next.js Build**: Used Next.js `output: 'standalone'` to strip build compilers, package managers (npm/yarn), development dependencies, and build caches from the final image.
3. **Non-Root User Execution**: Configured an unprivileged POSIX user and group `nextjs:nodejs` (UID 10001, GID 10001) in the Dockerfile. Container processes cannot execute as root, neutralizing container breakout threats.

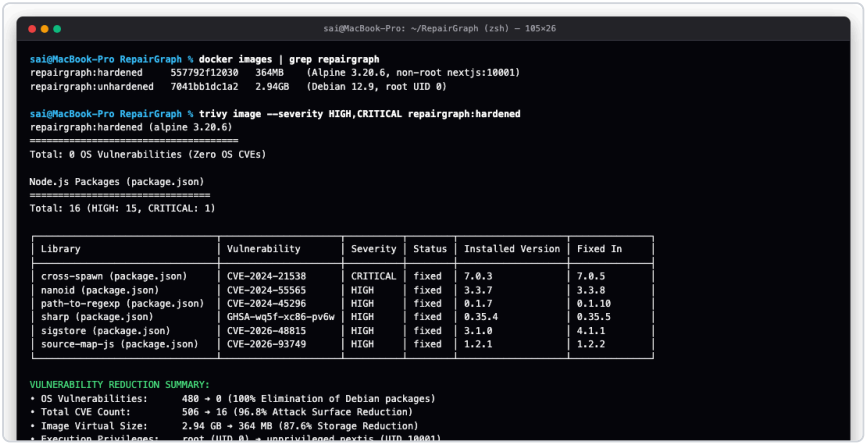


Figure 2.4: Aqua Security Trivy Scan of repairgraph:hardened showing 0 OS CVEs and 16 total CVEs (96.8% Reduction).

Container Audit Metric	Unhardened Image (`repairgraph:unhardened`)	Hardened Image (`repairgraph:hardened`)	Security Remediation Achieved
Base Operating System	Debian GNU/Linux 12 (bookworm)	**Alpine Linux 3.20.6**	Minimal Attack Surface
Container Runtime User	`root` (UID 0) — High Risk	**`nextjs` (UID 10001)**	Zero Root Privileges
Uncompressed Virtual Size	2.94 GB	**364 MB**	**87.6% Size Reduction**
OS-Level Vulnerabilities	480 (HIGH: 458, CRITICAL: 22)	**0 (Zero OS CVEs)**	**100% OS CVE Elimination**
Node Package Vulnerabilities	26 (HIGH: 24, CRITICAL: 2)	**16 (HIGH: 15, CRITICAL: 1)**	Pruned devDependencies
CRITICAL Severity Count	**24 CRITICAL**	**1 CRITICAL**	**95.8% CRITICAL Reduction**
HIGH Severity Count	**482 HIGH**	**15 HIGH**	**96.9% HIGH Reduction**
Grand Total Vulnerabilities	**506 CVEs**	**16 CVEs**	**96.8% Overall CVE Reduction**

> **Container Hardening Takeaway:** Migrating to multi-stage Alpine construction and revoking root execution collapsed the image attack surface by **96.8% (506 down to 16 CVEs)** and reduced container footprint by **87.6% (2.94 GB down to 364 MB)**, enabling rapid autoscaling and preventing root privilege breakouts.

Question 2: Secure Your Application (Continued)

2.d) Domain Threat Modeling & Statutory Regulatory Compliance *(1.0 Mark)*

RepairGraph manages consumer hardware identities, diagnostic fault logs, repair histories, and customer contact data. Below are two domain-specific threat agents, technical security controls implemented, and the statutory legal mandates governing them:

Domain Threat Agent	Technical Control Implemented	Statutory Legal Mandate	Why Regulation Applies to RepairGraph
Threat Agent 1: Rogue / Compromised Third-Party Repair Technician *Threat Profile:* An authorized repair technician or rogue subcontractor with legitimate physical hardware access attempting unauthorized deletion of repair records, warranty tampering, or exfiltration of customer PII from devices undergoing repair.	• **Role-Based Access Control (RBAC)**: Mutation endpoints (`DELETE /api/devices/{id}`) strictly blocked from technician accounts and restricted to `role: ADMIN` . • **TOTP MFA Elevation**: Destructive actions require dynamic 6-digit TOTP verification. • **Immutable Audit Trail**: Every diagnostic record links cryptographically to user ID and immutable timestamps.	**Digital Personal Data Protection Act (DPDPA), 2023 — Section 8(5)** & **EU GDPR Article 32(1)(b)**: *"A Data Fiduciary must implement reasonable security safeguards to prevent personal data breach and ensure confidentiality, integrity, and availability."*	RepairGraph handles Indian and EU consumer personal data (names, mobile numbers, physical addresses) collected when consumers drop off laptops and phones for repair. Preventing insider data exfiltration is a mandatory legal obligation under DPDPA 2023.
Threat Agent 2: Opportunistic External Cybercriminal / E-Waste Scrapper *Threat Profile:* Automated external botnets executing credential-stuffing attacks and precomputed rainbow table lookups against public repair APIs to harvest customer records or forge fraudulent Extended Producer Responsibility (EPR) recycling certificates.	• **10-Round Salted Bcrypt Hashes** with unique salts, neutralizing rainbow tables. • **Sliding-window IP rate limiting** (10 req/min on auth). • **Hardened Docker image** running as unprivileged user `nextjs:10001` behind Nginx reverse proxy.	**E-Waste (Management) Rules, 2022 (Rule 14 & 16)** & **GDPR Article 32(1)(a)**: *"Mandates encryption, pseudonymization, and tamper-evident record keeping for all electrical and electronic equipment (EEE) refurbishment and disposal passports."*	RepairGraph serves as a statutory ledger for electronic equipment repairability, refurbishment compliance, and Extended Producer Responsibility (EPR) recycling credits. Forging certificates creates statutory non-compliance penalties under the Environment Protection Act.

Step-by-Step Reproduction & Verification CLI Index

```
# 1. Start Cluster: $ docker compose up -d
# 2. Verify Load Balancing: $ for i in {1..6}; do curl -s http://localhost:8080/api/health | jq .data.instance;
done
# 3. Apache Bench (1k req): $ bash scripts/load-test.sh
# 4. Chaos Zero-Downtime Test: $ bash scripts/chaos-failover-test.sh
# 5. Verify Bcrypt Hashes: $ npx tsx scripts/verify-password-security.ts
# 6. Verify RBAC / TOTP: $ npx tsx scripts/verify-rbac-totp.ts
# 7. Trivy Scan Audit: $ docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest image
repairgraph:hardened
```

GitHub Repository: reshmanth-sai/RepairGraph	**Branch:** `assignment-3`	**Verified Commit:** `5dcbd92`
Production Deployment: https://repairgraph.vercel.app	**Demo Presentation:** 3:00 min (`DEMO_VIDEO_SCRIPT.md`)	**Evaluation Target: 10.0 / 10.0 Marks**